

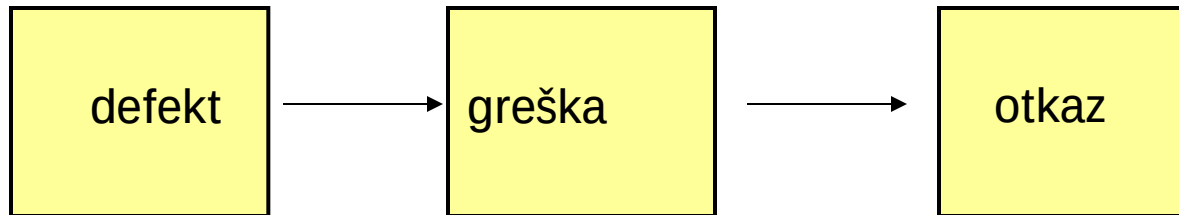
Distribuirani sistemi

Otpornost na greške (fault tolerance)

Otpornost na greške

- * Tolerantnost sistema na defekte (fault tolerance) predstavlja osobinu sistema da korektno funkcioniše uprkos pojavi grešaka.
- * Hardver, softver i mreže ne mogu biti 100% pouzdani
- * Računarski sistemi imaju široku primenu u svakodnevnom životu.
 - Veliki broj takvih sistema koristi se u hazardnim uslovima i za daljinsko upravljanje (u nuklearnim reaktorima, avionima i svemirskim brodovima)
 - U takvom okruženju računarski sistemi su, pre svega zbog uticaja zračenja, u velikoj meri podložni greškama.
 - što je sistem složeniji, to su šanse za nastupanje defekata veće
 - Zbog toga, pored performansi, otpornost na defekte postaje vrlo važno pitanje.
- * distribuirani sistemi mogu biti otporniji na defekte od centralizovanih sistema (kod kojih je otkaz najčešće fatalan), ali sa više hostova, verovatnoća individualnih defekata se povećava

Defekti (Faults), greške (Errors) i otkazi (Failures)



- * Defekt je trajni ili privremeni nedostatak koji se javlja u nekoj hardverskoj ili softverskoj komponenti
- * Greške su manifestacija defekta i predstavljaju odstupanje vrednosti podatka od očekivane (defekt je uzrok greške)
- * Otkaz nastupa kada sistem više ne može da obavlja funkciju za koju je projektovan
- * Za sistem se kaže da je otporan (tolerantan) na greške (ili defekte) ako je u stanju da korektno funkcioniše čak i u prisustvu grešaka

kategorizacija grešaka

* U odnosu na trajanje

● Prolazne (transient faults)

➤ Pojave se jednom i nestanu

- Npr. Istekao time out za prenos poruke, ali nakon ponovnog slanja poruka je uspešno otposlata

● Peridične (intermittent)

➤ Greška se pojavi, zatim nestane, pa ponovo pojavi. Najnezgodniji tip grešaka

- Npr greška u mrežnom kablju kod koga postoji neki prekid

● Stalne (permanent)

➤ Greške koje postoje sve dok se komponenta ne zameni ispravnom

- Greška u hard disku, pregorela komponenta, bagoviti sw.

* Druga klasifikacija grešaka

- Tvz. “mirna” (fail-silent, ili fail-stop) greška– komponenta prestaje sa radom i ne generiše nikakv izlaz ili generiše poruku o grešci
- Vizantijske greške – komponenta nasatavlja sa radom i generiše pogrešan rezultat.

Kako sistem učiniti otpornim na greške?

✱ Opšti prilaz u projektovanju sistema otpornih na greške je redundansa.

- Može se primeniti na nekoliko nivoa

- Informaciona

- Otpornost na greške se postiže repliciranjem ili kodiranjem podataka (npr. Hammingovi kodovi, ABFT)

- Vremenska

- Tolerantnost na greške se postiže izvršenjem operacija više puta u vremenu. Npr. timeouti i retransmisija poruka.
- Ova vrsta redundantnosti je korisna u slučaju prolaznih ili periodičnih grešaka.

- Fizička redundansa

- hardverska
 - » Bavi se uređajima a ne podacima. Hardver se umnožava da bi sistem mogao tolerisati otkaze. Npr. RAID diskovi (redundant array of independent disks) i backup servri.
- softverska
 - » replicirani procesi

Koliko otpornosti na greške?

- * 100% tolerantnost na greške se nikada ne može postići.
 - Što smo bliži cifri od 100%, to je sistem skuplji i složeniji.
- * Za sistem se kaže da je k -tolerantan na greške ako može podneti k grešaka a da pri tome nastavi da korektno funkcionise.
 - Ako je reč o “mirnim” greškama potrebno je $k+1$ komponenta da bi sistem i dalje funkcionisao
 - Ako k otkaže, postoji još uvek jedna koja korektno radi
 - Ako je reč o Vizantijskim greškama potrebno je $2k+1$ komponenti
 - k može generisati pogrešne rezultate, ali će $k+1$ korektno funkcionisati i većinskim glasanjem izglasati korektan rezultat.

Primer1: fizička hardverska redundansa

* Tehnika za postizanje visoke pouzdanosti kroz fizičku redundantnost.

- Najjednostavnija varijanta- TMR (Tripple Modular Redundancy)
- Koriste se tri primerka komponente da bi se detektovala i korigovala jednostruka greška.
- Primer sistem bez redundanse



Otkaz bilo koje komponente, dovodi do otkaza celog sistema

Ako komponenta i ima pouzdanost R_i , pouzdanost celog sistema je

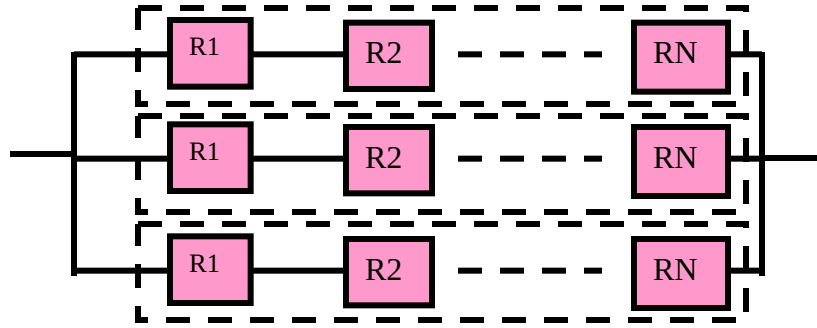
$$R = R_1 R_2 R_3 \dots R_N = \prod_{i=1}^N R_i$$

Npr. ako sistem ima 100 komponenti, otkaz bilo koje komponente će izazvati otkaz celog sistema. Ako svak komponenta ima pouzdanost 0.999, pouzdanost celog sistema je

$$R = R_1 R_2 R_3 \dots R_{100} = (0.999)^{100} = 0.905$$

fizička hardverska redundansa (nast.)

- * ako hardver repliciramo 3 puta, pouzdanost sistema će se povećati



- * sada postoje tri paralelna puta.
- * ako je pouzdanost na jednom putu R_i , a nepouzdanost $Q_i = 1 - R_i$, tada je nepouzdanost celog sistema

$$Q = Q_1 Q_2 Q_3 \dots Q_N$$

tj.

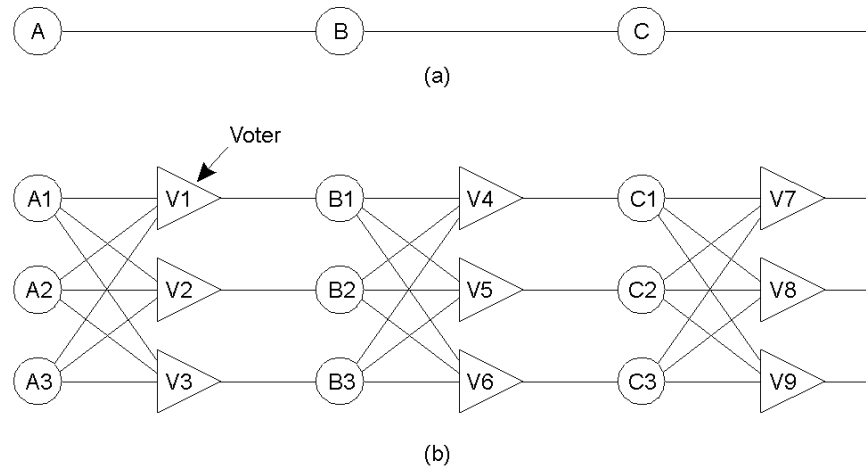
$$1 - R = [1 - R_1][1 - R_2][1 - R_3] \dots [1 - R_N]$$

Za prethodni primer gde je pouzdanost sistema iznosila 0.905, nakon tripliciranja, pouzdanost iznosi

$$R = 1 - (1 - 0.905)^3 = 0.9991$$

fizička hardverska redundansa (nast.)

* TMR tehnika se primenjuje u sprezi sa glasačima (voterima)



a) Sistem bez redundantnosti: signal prolazi kroz uređaje A, B i C sekvencijalno. Ako je jedan od uređaja neispravan, konačni rezultat će verovatno biti nekorektan.

b) TMR: svaka komponenta je replicirana 3 puta. Izlaz iz svake komponente se vodi na glasač (voter). I voteri su replicirani. Ako su tri ili dva ulaza ista, izlaz je jednak ulazu. Ako su tri ulaza različita, izlaz je nedefinisan.

Primer2: informaciona redundansa – ABFT tehnika (Algorithm base fault tolerance)

- * tehnika koja se može primeniti kod odedjene klase algoritama
 - tipično za matrična izračunavanja (množenje matrica, LU dekompozicija, inverzija..)
- * Primer: množenje matrica $C=A*B$, A , B , C su dimenzija $n \times n$
 - Matrica A se proširuje vrstom čiji elementi predstavljaju sumu elementa odgovarajuće kolone (matrica A_c dimenzija $(n+1) \times n$)
 - Matrica B se proširuje kolonom čiji elementi predstavljaju sumu elemenata odgovarajuće vrste (matrica B_r dimenzija $n \times (n+1)$).
 - Rezultujuća matrica C_f ima je dimenzija $(n+1) \times (n+1)$

$$C = A * B, \Rightarrow C_f = A_c * B_r$$

$$A_c = \begin{bmatrix} A \\ \vec{e}^T A \end{bmatrix}, \quad B_r = \begin{bmatrix} B & B\vec{e} \end{bmatrix}, \quad C_f = \begin{bmatrix} AB & AB\vec{e} \\ \vec{e}^T AB & \vec{e}^T AB\vec{e} \end{bmatrix}$$

$$\vec{e}^T = [1 \ 1 \ 1 \dots 1]$$

Primer2 (nast.)

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad A_c = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ \hline 2 & 2 \end{bmatrix}$$
$$B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad B_r = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 1 & 2 \\ \hline 1 & 1 & 2 \end{bmatrix}$$
$$C_f = A_c * B_r = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix} * \begin{bmatrix} 1 & 1 & 2 \\ 1 & 1 & 2 \\ \hline 1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 4 \\ 5 & 2 & 4 \\ \hline 4 & 4 & 8 \end{bmatrix}$$

Npr. Element c_{21} je pogrešan i ima vrednost 5 umesto 2. izračunavanjem sume elementa prve kolone (druge vrste) i poređenjem sa kontrolnom sumom odgovarajuće vrste i kolone detektovaće se i locirati greška.

Korekcija greške:

$$C_{12} = 5 + (4 - 7) = 5 - 3 = 2$$

Detekcija greške: naći sume elemenata svake vrste / svake kolone (S1) i uprediti ih sa dobijenim kontrolnim sumama (S2); ako postoje razlike detektovana je greška

Lokacija greške: u preseku vrste i kolone gde je otkriveno neslaganje kontrolnih suma i suma elemenata

Korekcija greške: $c := c' + (S2 - S1)$

Erasure coding

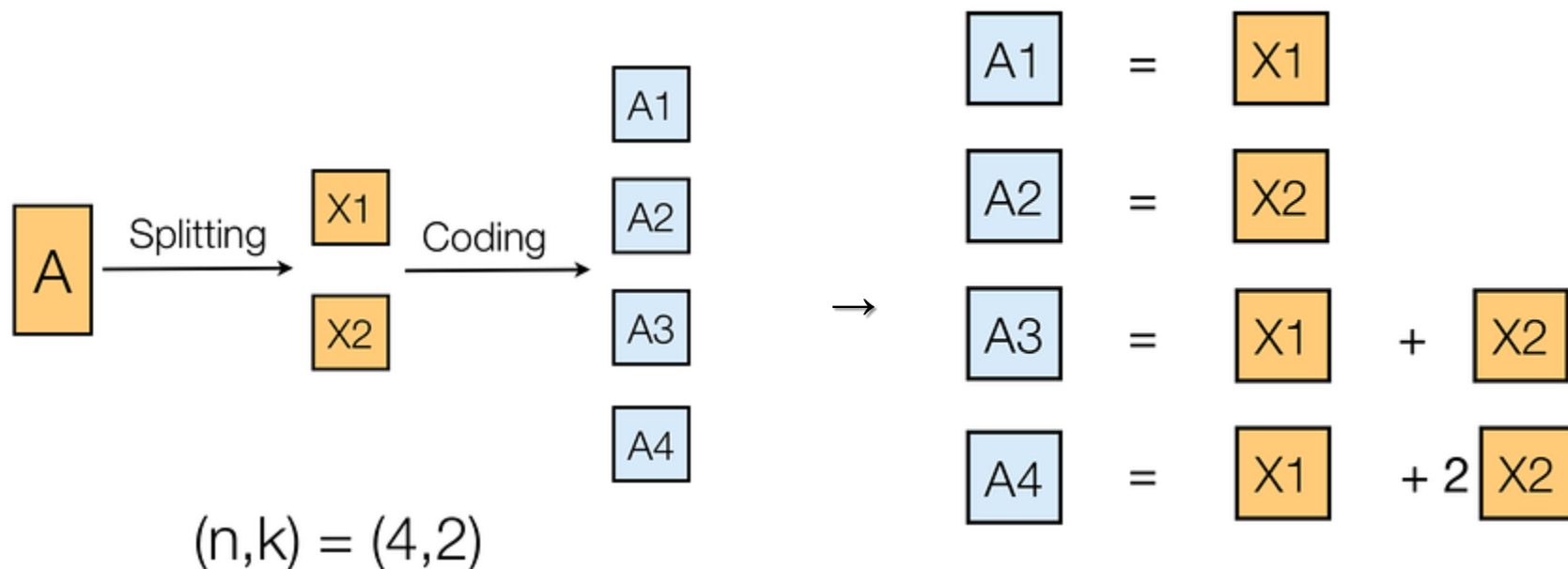
- * Tehnika za postizanje otpornosti na greške (fault tolerance)
- * Tehnika kodiranja kod koje se skup od k podataka kodira skupom od n , $n > k$ podataka tako da se na osnovu bilo kog skupa od k podataka mogu regenerisati originalni podaci.
 - (n,k) erasure code omogućava tolerisanje $n-k$ grešaka.
 - Tehnika koja je široko prihvaćena kod HDF (Hadoop file system)
 - Hadoop je sistem za skladištenje, pretraživanje i procesiranje velike količine podataka (big data)

Erasure coding - primer upotrebe u DFS

- * Posmatrajmo fajl veličine M
- * Podeliti fajl na k blokova veličine M/k
- * Primeniti (n, k) code na ovih k blokova da bi se dobilo n blokova, svaki veličine M/k .
 - Na taj način fajl je proširen n/k puta
 - Potrebno je da n bude veće ili jednako k .
 - Ako je n jednako k , fajl je samo podeljen na blokove, ali nema nikakvog kodiranja
- * Bilo koji podskup k od n blokova se može uzeti da bi se rekonstruisao fajl
 - to znači da kod može tolerisati $(n-k)$ grešaka

Primer(4,2) koda

- * Podelimo fajl na 2 bloka i formiramo 4 nova bloka
- * Neka se 4 bloka pamte u 4 različita čvora



Zašto je ovo bolje nego da smo jednostavno svaki blok zapamtili u 2 različita čvora?

Primer(4,2) koda

X1

X2

X1

X2

- Šta ako čvorovi 1 i 3 otkažu?
 - Fajl se ne može regenerisati!

Node 1

Node 2

Node 3

Node 4

A1

A2

A3

A4

Moguće je
regenerisati fajl

Node 1

Node 2

Node 3

Node 4

$$A1=X1$$

$$A2=X2$$

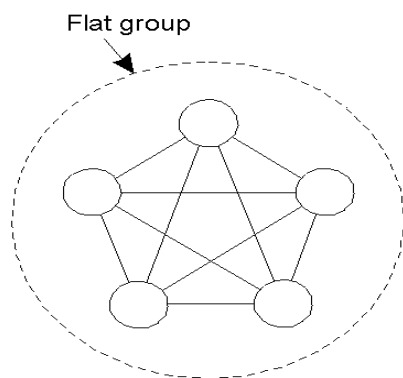
$$A3=X1+X2$$

$$A4=X1+2*X2$$

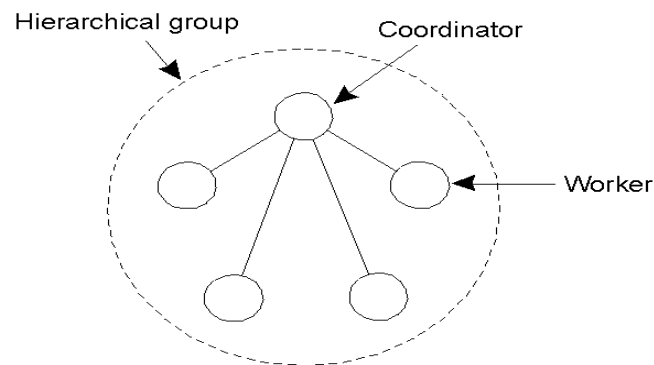
Kako se tolerantnost na greške postiže u DS?

* Kroz replikaciju

- nekoliko identičnih kopija procesa se organizuje u grupu.
- Kada se neka poruka pošalje grupi, svi članovi grupe je primaju
 - ako jedan proces u grupi otkáže (greši) postoji drugi koji obavlja isti posao
- replicirani procesi mogu biti oraganizovani u grupe ravnopravnih procesa (flat) ili hijerarhijske grupe
 - ravne grupe – svi procesi su jednaki
 - hijerarhijske – postoji jedan koordinator



(a)



(b)

Kako se tolerantnost na greške postiže u DS? (nast.)

* Replikacija može biti bazirana na postojanju primarne i backup kopija (pasivna replikacija)

- koristi se kod hijerarhijski organizovanih grupa
 - Primarni server obavlja ceo posao
 - Ako primar otkaže, jedan od backup servera preuzima posao.
 - Otkaz primara treba da je nevidljiv za aplikativne programe
- Kako backup zna da je primar otkazao
 - Periodično mu šalje poruku “da li si živ?”
 - Ako istekne timeout i na pitanje se ne dobije odgovor, backup server preuzima posao.

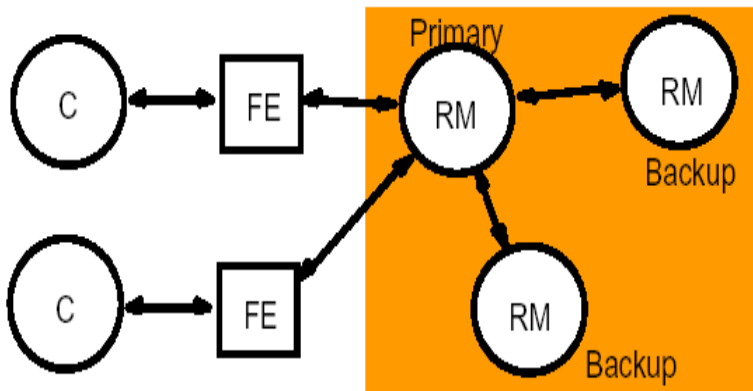
* aktivna replikacija

- koristi se kod grupa ravnopravnih procesa
 - klijentski zahtev se prosledjuje svim procesima u grupi
 - Zahtevi moraju da stignu u sve servere u istom redosledu, tj. neophodna je sinhronizacija
 - Ako se zahtevi procesiraju po redosledu, svi serveri koji korektno funkcionišu će dati isti odgovor.

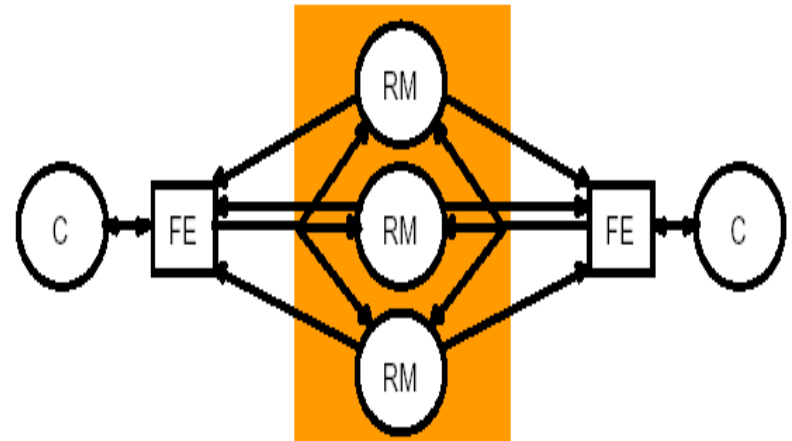
Pasivna i aktivna replikacija

* Pasivna replikacija

- Klijent (C), Front End (FE) = klijent interface, Replica Manager (RM) = davalac usluge (server)



* Aktivna replikacija



* Prednosti replikacije bazirane na postojanju primarne kopije u odnosu na aktivnu replikaciju

- Jednostavniji dizajn:

- dovoljan je samo jedan backup server
- Nema potrebe za grupnom komunikacijom (multicast) i nema problema sa sinhronizacijom (poruka se šalje samo jednom serveru)

* Nedostaci

- Loše funkcioniše u prisustvu Vizantijskih grešaka – primar može pogrešno da radi a da greška ne bude otkrivena

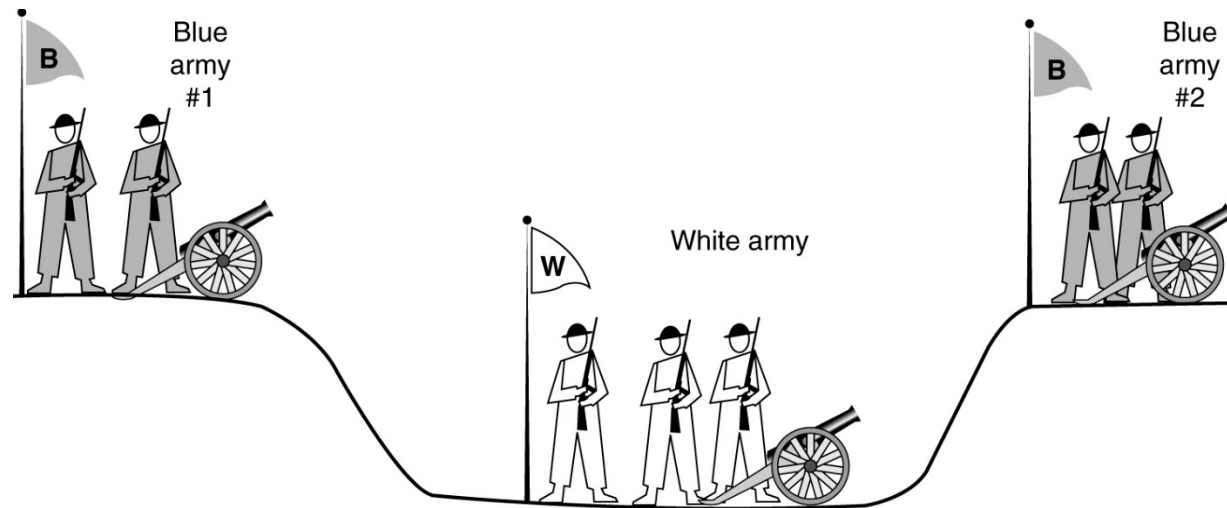
- jedini način za postizanje tolerantnosti na greške vizantijskog tipa je kroz aktivnu replikaciju

Usaglašavanje u prisustvu grešaka

- * Distribuirani procesi često moraju da se usaglaise oko nečega.
 - Npr. Izbor koordinatora, okončanje transakcije, pristup kritičnoj sekciji, ili šta je korektan rezultat, itd.
- * Kako postići konsenzus ako neki procesi u grupi nekorektno funkcionišu ili je komunikacioni kanal nepouzdan?
- * Usaglašavanje je potrebno postići u konačnom broju koraka.
- * Posmatraju se dva odvojena scenarija:
 - Ispravni procesori, a nesavršeni komunikacioni kanali (“Problem dve armije”)
 - Nesavršeni procesori, a pouzdani komunikacioni kanal (“Problem vizantijskih generala”)

Problem dve armije (two army problem)

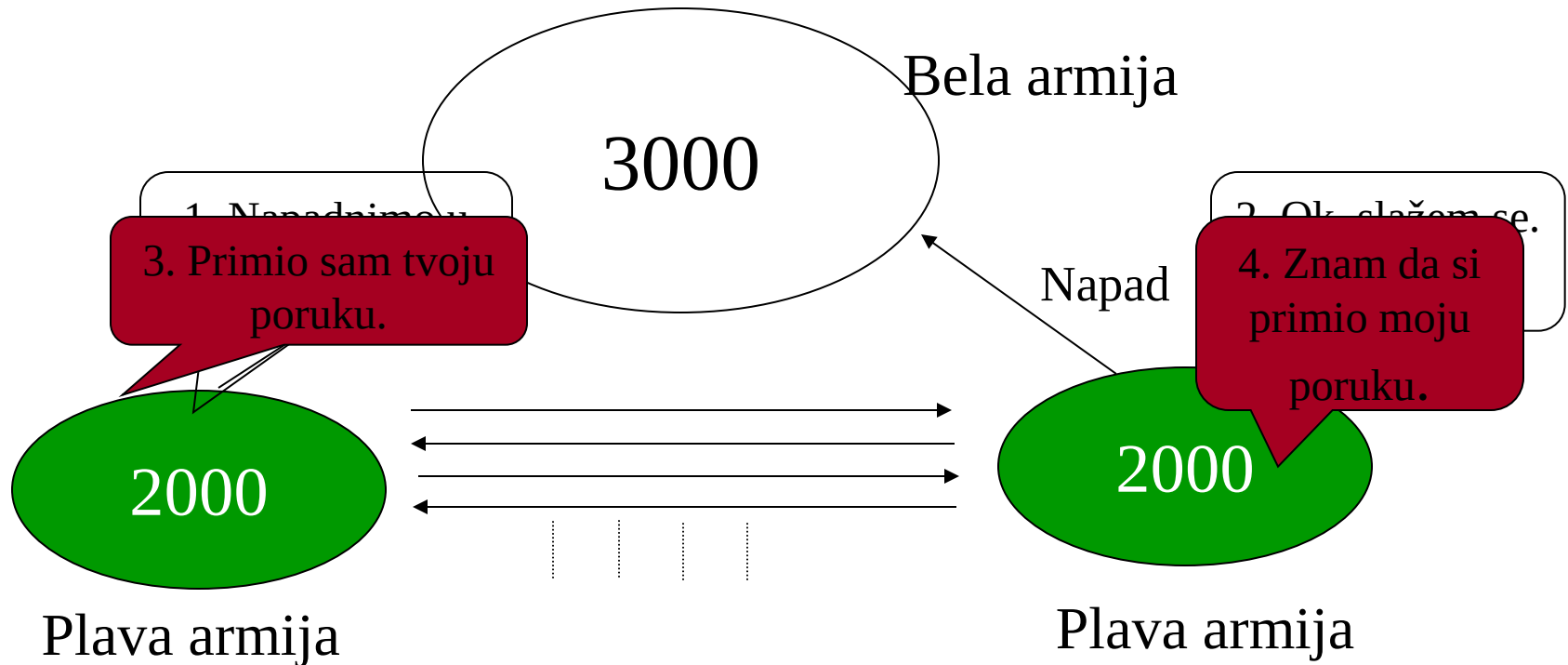
- * Dve divizije, A i B, jedne armije, treba da koordinišu napad na drugu armiju, C.
- * A i B imaju po 2000 vojnika, a armija C 3000.
 - Ako se A i B dogovore da zajedno napadnu, mogu da pobeđe C



Problem dve armije (nast.)

- * A i B su fizički odvojene i koriste kurira za razmenu poruka.
 - A šalje poruku B u kojoj piše “izvršimo napad u zoru”.
 - B prima poruku i slaže se sa predlogom (šalje ok – potvrdu)
 - Kurir stiže u A sa povratnom porukom (potvrdom), ali tada A zaključuje da B ne zna da li je A primio potvrdu i neće biti siguran da li da obave napad.
 - A može da odluči da pošalje poruku B-u “primio sam potvrdu”, ali tada A neće biti siguran da li je ta poruka stigla u B, itd.
- * Problem je poznat pod nazivom problem višestrukih potvrda (*multiple acknowledgment problem*).
- * On pokazuje da čak i u prisustvu ispravnih procesora, usaglašavanje između dva procesa nije moguće ako su komunikacije nepouzidane.
 - Jedino se možemo nadati da najčešće jesu!
- * Ne postoji protokol koji može da reši problem nepouzanih kanala!

Problem dve armije



→ Može se pokazati da generali plave armije nikada ne mogu postići konsenzus (zbog nepouzidane komunikacije)

Problem vizantijskih generala

- * Pouzdane komunikacije, nepouzdani procesori.
- * Postoji n generala svaki sa svojom divizijom i treba da donesu odluku da li da napadnu neprijatelja ili ne.
 - Komunikacije su pouzdane (radio ili telefon), ali m generala su izdajnici i pokušavaju ostale generale da spreče da postignu konsenzus šaljući im netačne i kontradiktorne informacije.
- * Pitanje je da li lojalni generali ipak mogu postići konsenzus (doneti istu odluku) ?
 - Svaki general daje svoj predlog za akciju: napad (N) ili povlačenje (P) šaljući poruku svim ostalim generalima, i prima poruke od ostalih generala sa njihovim predlozima
 - generali izdajnici mogu slati različite poruke ostalim generalima
 - Svaki general donosi odluku o tome koju će akciju preduzeti većinskim izglasavanjem primljenih predloga uzimajući u obzir i svoj predlog
 - Kako da svi lojalni generali izglasaju isti predlog (napad ili povlačenje)?

Lamportov algoritam za problem vizantijskih generala

- * Lamport je predložio rešenje koje radi pod određenim uslovima.
- * Zaključak je da bilo koje rešenje za prevazilaženje problema m generala izdajnika zahteva najmanje $3m+1$ učesnika ($2m+1$ lojalnih generala), tj. Više od $2/3$ generala mora biti lojalno.
- * Pokazano je da je potrebno obaviti $m+1$ rundu razmene poruka i razmeniti $O(mn^2)$ poruka
- * Ovo rešenje je veoma skupo.
- * Ima ga smisla primeniti u nekom hardveru specijalne namene, ali je neefikasno ga koristiti u distribiranom računarskom okruženju.

BGP - problem

- * Problem n vizantijskih generala se može posmatrati kao n identičnih problema
 - postoji jedan general i $n-1$ poručnik
 - m poručnika (ili $m-1$ poručnik i general) mogu biti izdajnici
 - lojalni general šalje svim poručnicima istu poruku (plan akcije: napad ili povlačenje)
 - svaki poručnik primljenu poruku prosleđuje ostalim poručnicima (tj. ponaša se kao general)
 - kada se sve poruke prikupe primenjuje se većinsko glasanje
 - ako je moguće izglasati vrednost, ona se izglasa, inače se koristi podrazumevana akcija (npr. povlačenje)
- * Želimo da postignemo sledeće:
 1. svi lojalni poručnici moraju da donesu istu odluku
 2. ako je general lojalan, tada svi lojalni poručnici poštuju njegovo naređenje.

Byzantine Generals Problem (BGP)

Mora se postići da

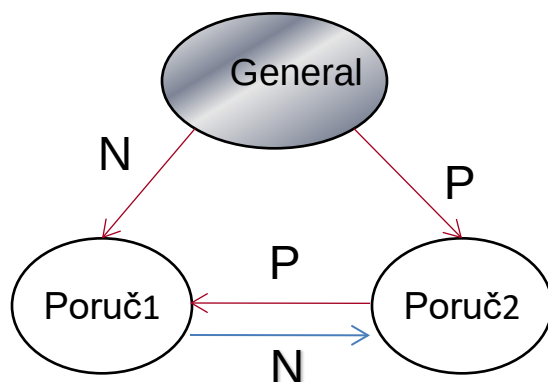
1. Svi lojalni poručnici donesu istu odluku
2. ako je general lojalan, tada svaki lojalni poručnik poštuje generalovo naređenje

Pretpostavke:

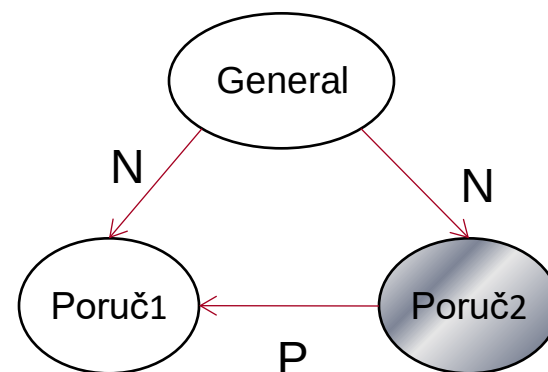
1. Svaka poruka se prima korektno (komunikacioni kanal ne unosi greške)
2. Prijemnik zna ko je posalo poruku
3. Odsustvo poruke se može detektovati

• poručnik 2 je izdajnik

general je izdajnik



N – napad
P - povlačenje

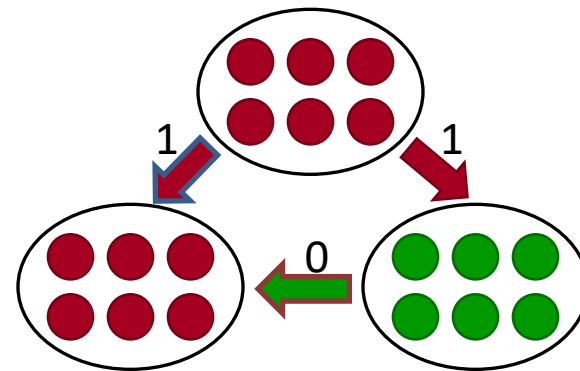
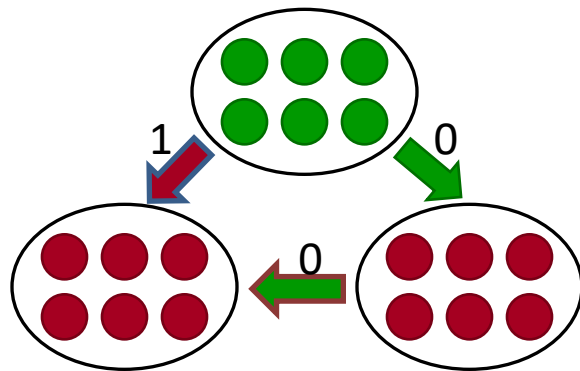


lojalni poručnici će doneti istu odluku

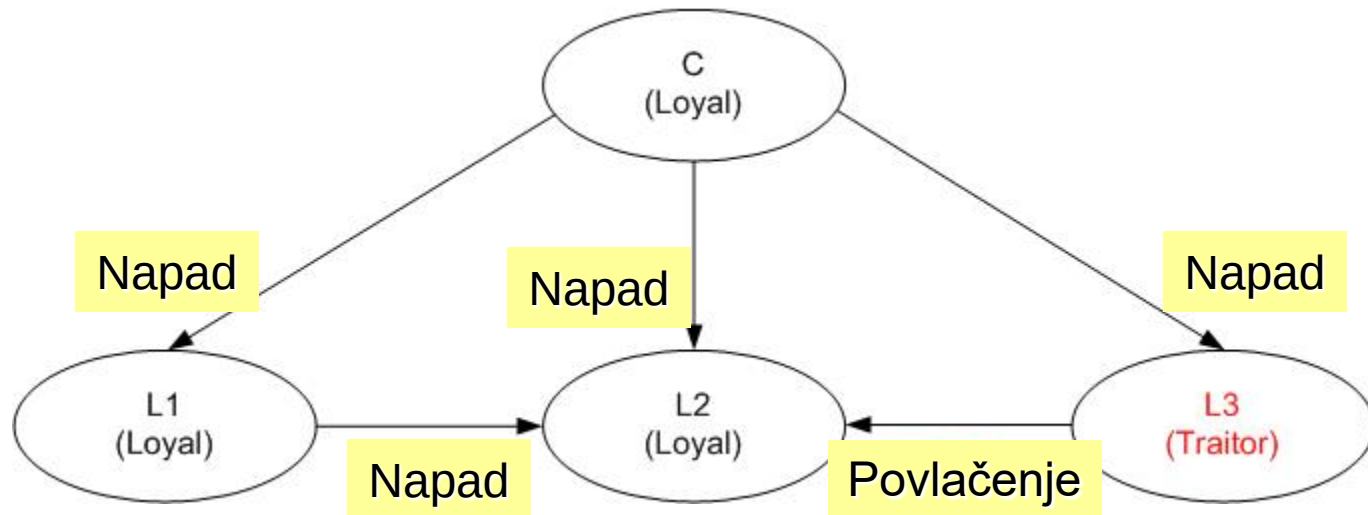
• lojalni poručnik 1 neće poslušati naređenje lojalnog generala!

BGP

- * ako postoji m izdajnika i $n \leq 3m$ generala nemoguće je postići konsenzus
- * da bi moglo da se toleriše m izdajnika mora postojati najmanje $2m+1$ lojalnih generala, tj minimalni broj generala mora biti $n=3m+1$



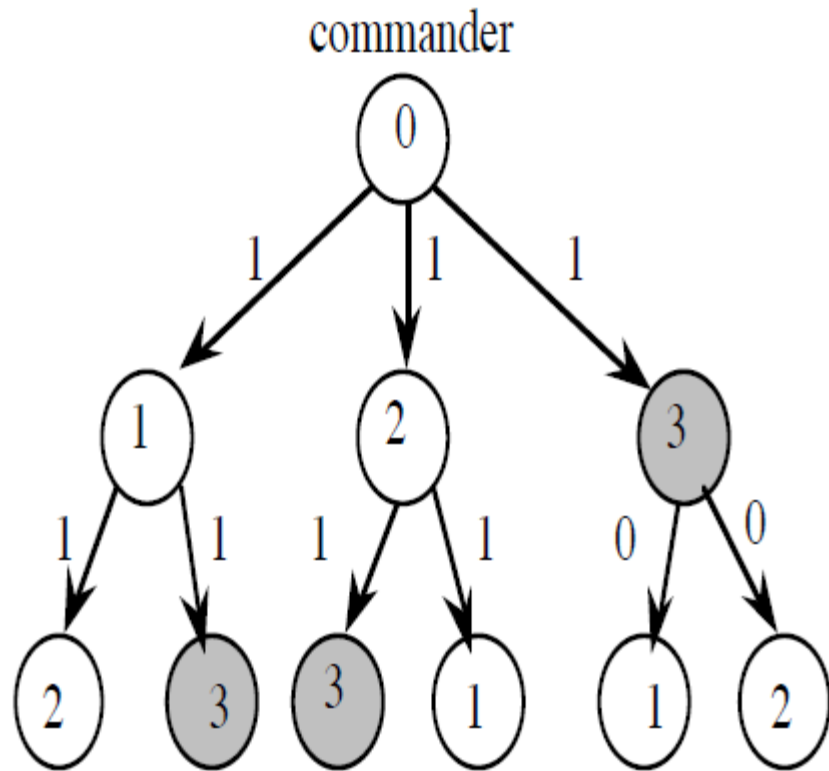
Primer $n=4$, $m=1$



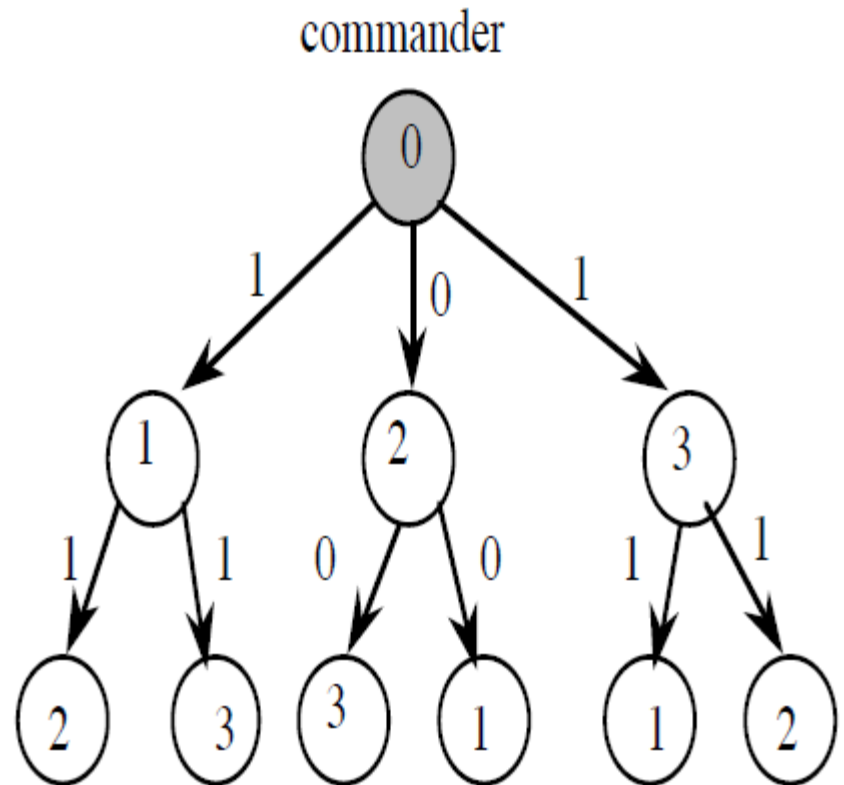
* $n=4$ generala; $m=1$ izdajnik

* L2 računa $\text{majority}(\text{Napad}, \text{Napad}, \text{Povlačenje}) = \text{Napad}$

BGP- ekvivalentni prikaz rešenja



(a)



(b)

Example with $m=1$ and $n=4$

Lamportov rekurzivni algoritam $OM(m)$

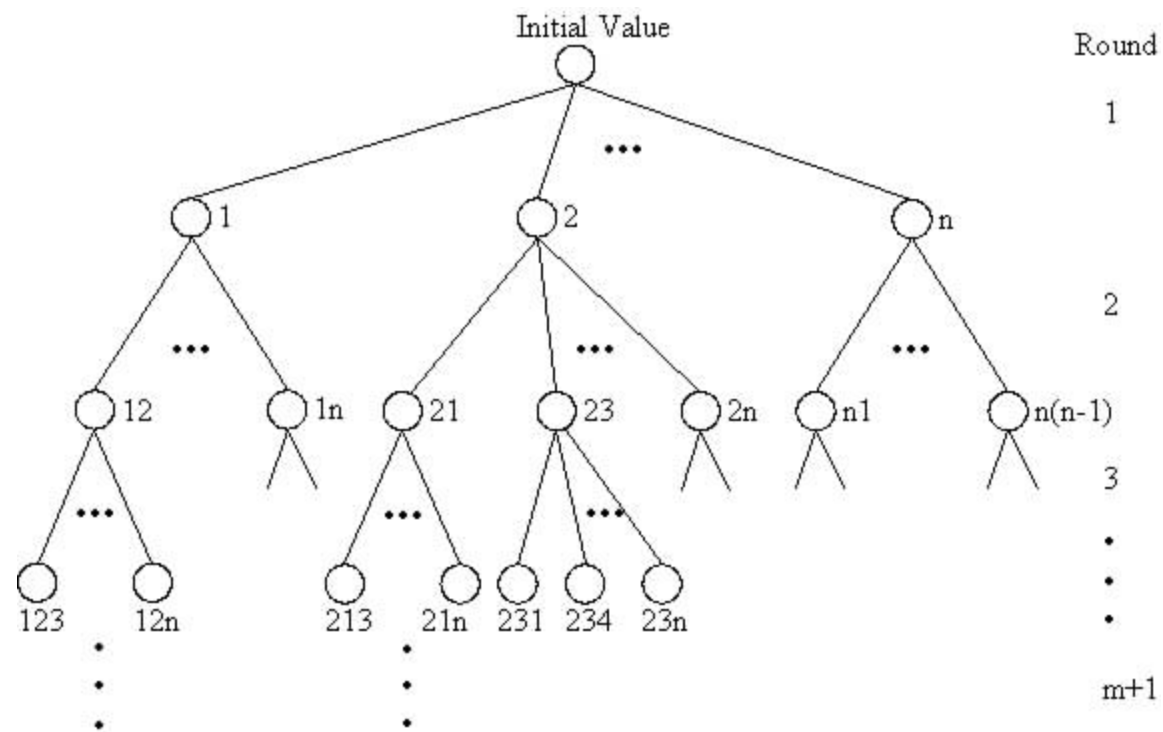
Rekurzivna definicija: slučaj $m=0$ okončava rekurziju

Algoritam $OM(0)$ – (pošto je $m=0$ nema izdajnika)

1. General šalje svoje naređenje svim poručnicima
2. Svaki poručnik poštuje naređenje koje je dobio od generala

Algoritam $OM(m)$, $m > 0$

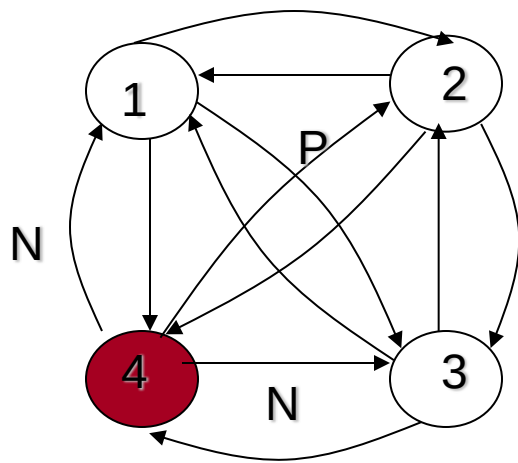
1. General šalje svoje naređenje svakom poručniku
2. Za svako i , neka v_i označava vrednost koju je poručnik i primio od generala. Poručnik i deluje kao general u Algoritmu $OM(m-1)$ da bi poslao vrednost v_i ostalim $n-2$ poručnicima.
3. Za svako i , i svako $j \neq i$, neka v_j predstavlja vrednost koju je poručnik i primio od poručnika j u koraku 2 (korišćenjem $OM(m-1)$). Poručnik i izračunava $\text{majority}(v_1, \dots, v_{n-1})$.



primer1: 4 generala, 1 izdajnik

- * 3 lojalna generala i $m=1$ izdajnik
- * potrebno je obaviti $m+1=1+1=2$ runde razmene poruka da bi se postigao konsenzus
- * u prvoj rundi svaki general šalje svim ostalim generalima svoj predlog
- * u drugoj rundi svaki general šalje ono što je primio od drugih generala
 - lojalni generali prosledjuju tačno poruke koje su primili od drugih generala
 - izdajnici mogu promeniti poruke
 - kada se prikupe sve poruke primenjuje se većinsko glasanje da bi se odredila veličina trupe svakog generala i donela odluka

primer1: 4 generala, 1 izdajnik (nast.)



I runda

1. predlaže N
2. predlaže N
3. predlaže P
4. šalje predlog N 1. i 3. generalu, a 2. predlog P

1. general ima predloge (N,N,P,N)
2. general ima predloge (N,N,P,P)
3. general ima predloge (N,N,P,N)
4. general ima predloge (N,N,P,*)

LOJALNI GENERALI NE MOGU POSTIĆI KONSENZUS
POSLE PRVE RUNDE!!

II runda:

1. prima
(N,N,P,P)
(N,N,P,N)
(* , * , * , *)
(N,N,P,*)

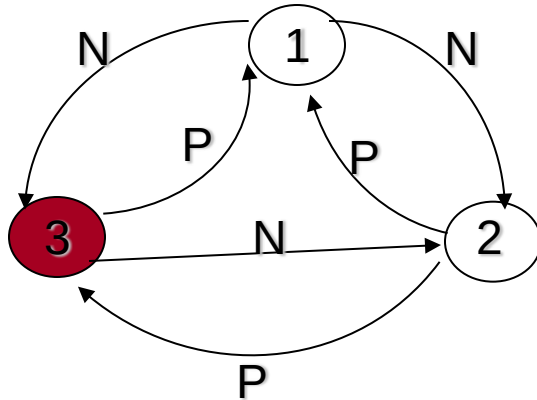
2. prima:
(N,N,P,N)
(N,N,P,N)
(* , * , * , *)
(N,N,P,*)

3. prima:
(N,N,P,N)
(N,N,P,P)
(* , * , * , *)
(N,N,P,*)

4. prima
(N,N,P,N)
(N,N,P,P)
(N,N,P,N)

(N,N,P,*) – svi lojalni generali imaju isti vektor i
mogu većinski izglasati istu odluku

primer2: 3 generala, 1 izdajnik



I runda

- general ima predloge (N,P,P)
- general ima predloge (N,P,N)
- general ima predloge (N,P,*)

II runda

1. prima vektore (N,P,N) (*,*,*)
2. prima vektore (N,P,P) (*,*,*)
3. prima (N,P,P) (N,P,N)

(?,?,?)

(?,?,?) – lojalni generali ne mogu postići konsenzus!

Broj lojalnih generala (korektnih procesa) mora biti veći od 2/3 ukupnog broja da bi se većinskim glasanjem postigao konsenzus.

Strategije za oporavak od greške

- * Kada nastupi greška, potrebno je izvršiti oporavak od greške i vratiti sistem u korektno stanje
- * To se može postići na dva načina:
- * Povratak u stanje pre nastupanja greške (Backward Recovery):
 - vratiti sistem u neko prethodno korektno stanje korišćenjem tačaka provere (checkpoints), a zatim nastaviti sa radom
 - npr retransmisija u slučaju gubitka paketa ili greške u primljenom paketu
- * Prelazak u novo stanje (Forward Recovery):
 - prevesti sistem u korektno novo stanje nakon čega se može nastaviti sa radom
 - Erasure coding, (n,k) –tehnika kodiranja podataka koja omogućava korekciju u slučaju da je nastupila greška (skup od k poruka se kodira u n poruka tako da bilo kojih k poruka na odredištu mogu da se iskoriste da se rekonstruiše k originalnih poruka).

Forward i Backward Recovery

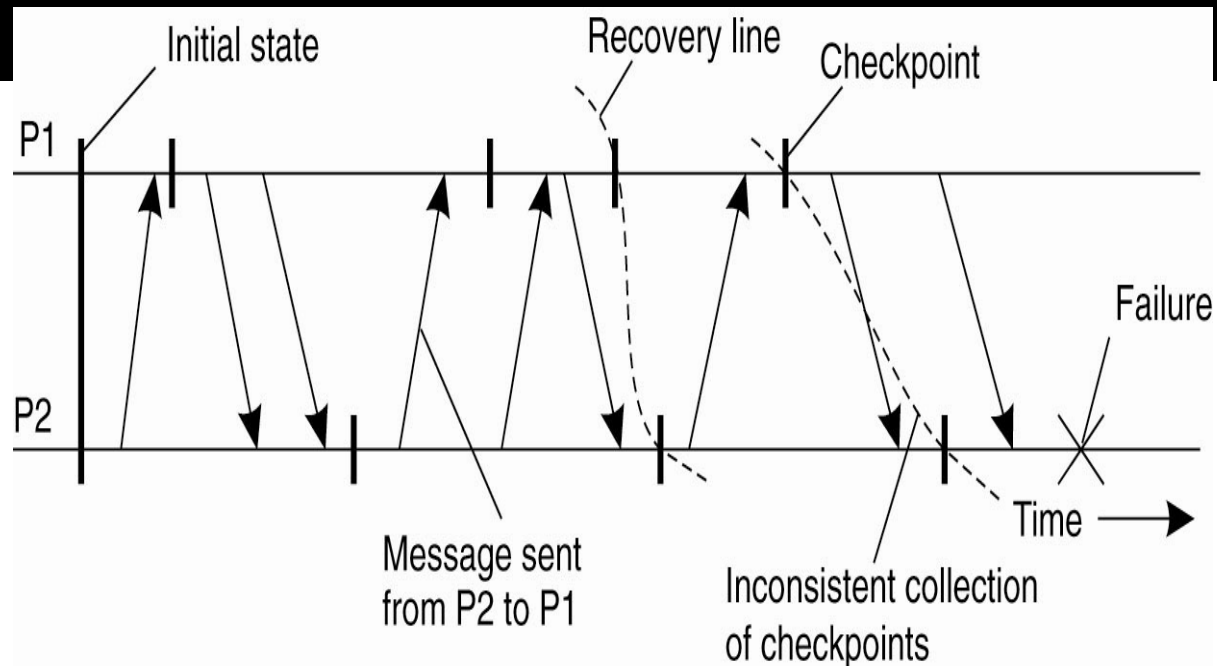
* Nedostatak oporavka sa povratkom u nazad

- beleženje stanja sistema ubacivanjem tačaka provere (Checkpointing) može biti skupo (pogotovo ako su greške retke)
 - uprkos ceni, ova tehnika se češće koristi od forward.
 - “logging” se može smatrati jednom vrstom checkpoint-a

* Forward Recovery:

- da bi ova tehnika funkcionisala neophodno je predvideti sve potencijalne greške
- kada nastupi greška, mehanizam za oporavak zna šta treba da uradi da bi doveo sistem u naredno korektno stanje.

Checkpointing



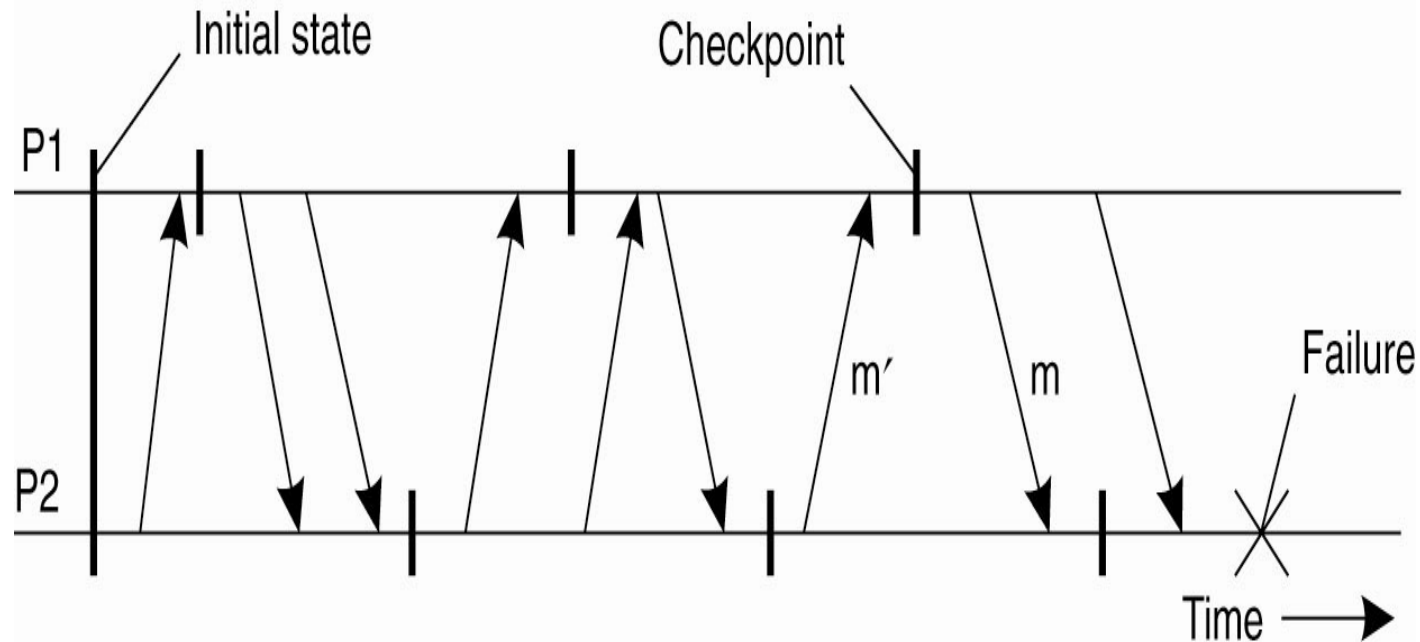
- Svaki proces povremeno pamti svoje stanje na disku (npr. vrednosti promenljivih, primljene poruke, poslate poruke) u tačkama provere.
- Ako nastupi greška, proces se vraća u stanje pre nastupanja greške na osnovu podataka zapamćenih u checkpoint.
- U DS potrebno je da se svi procesi vrate u stanje odakle je moguće izvršiti oporavak, tj. potrebno je pronaći liniju oporavka (konzistentni presek). Skup tačaka provere C je konzistentan ako za sve događaje e i e' važi

$$(e \in C) \wedge (e' \rightarrow e) \Rightarrow e' \in C$$

- to znči da ako je npr. proces P zabeležio prijem poruke iz procesa Q , tada u skupu mora postojati i tačka u kojoj je proces Q zabeležio slanje poruke procesu P (poruka je morala da stigne od nekog).

domino efekat kod nezavisnog beleženja stanja (checkpointing)

Međutim, ako svaki proces za sebe donosi odluku kada da kreira checkpoint, problem pronalaženja linije oporavka postaje veoma složen, čak može dovesti do tzv. domino efekta, tj. do vraćanja u polazno stanje.



Domino efekat- greška u P2 zahteva povratak u prethodno stanje i procesa P2 i procesa P1, ali checkpoint-i nisu konzistentni: P2 je zabeležio prijem poruke m, ali P1 nije zabeležio slanje poruke m. Zato se P2 mora vratiti još u nazad, ali sada imamo da je P1 zabeležio prijem poruke m' a P2 nije zabeležio slanje m' itd.

koordinisano beleženje stanja

- * svaki proces kreira checkpoint nakon globalno koordinisane akcije.
- * 2-phase blockirajući protokol
 - Koordinator šalje svim procesima u grupi poruku *checkpoint_REQUEST*
 - Svi procesi primaju poruku, kreiraju checkpoint, prestaju sa slanjem aplikativnih poruka, i šalju koordinatoru poruku *checkpoint_ACK*
 - Kada prikupi potvrde od svih procesa, koordinator šalje poruku *checkpoint_DONE* svim procesima, nakon čega se procesi deblokiraju i nastavljaju sa radom.
 - Zapamćeno stanje u svim procesima je konzistentno
 - izbegnut je domino efekat

Klijent-server komunikacija sa RPC

- * Cilj RPC je da sakrije komunikaciju tako što pokušava da poziv udaljene procedure izgleda kao poziv lokalne
 - dok nema grešaka u sistemu ovo funkcioniše dobro
 - ako nastupe greške nije uvek moguće sakriti razlike izmedju poziva udaljene i lokalne procedure.
- * Greške koje mogu nastupiti u RPC sistemu
 - klijent nije u stanju da locira server
 - zahtev upućen od klijenta ka serveru je izgubljen
 - otkaz servera nakon prijema zahteva od klijenta
 - odgovor servera klijentu izgubljen
- * svaka od ovih kategorija grešaka izaziva različite probleme i zahteva različita rešenja

Klijent nije u stanju da locira server

* Razlog može biti otkaz servera ili neodgovarajuća verzija klijent stub-a

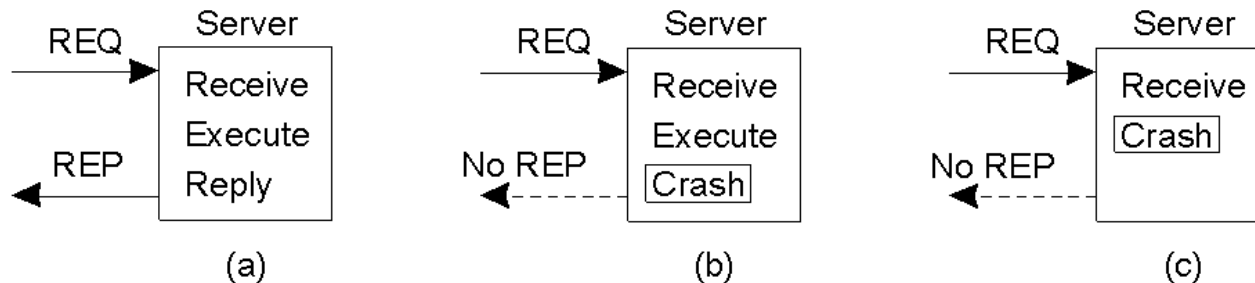
- novija verzija interfeisa i stub funkcija je instalirana na serveru
 - kada klijent pozove RPC binder neće biti u stanju da locira server
- neophodno je da kada se desi takva greška da se emituje exception
 - programer mora da napiše specijalnu proceduru koja se poziva kada se detektuje određena greška – transparentnost distribucije više ne postoji

* zahtev upućen od klijenta ka serveru je izgubljen

- najlakše se rešava

- klijent treba da startuje časovnik kada uputi zahtev
- ako istekne time out pre nego što stigne odgovor ili potvrda, vrši se retransmisija
- ako zahtev nije bio izgubljen, onda je ili server otkazao ili je izgubljen odgovor od servera $\Rightarrow$ server mora biti u stanju da prepozna retransmisiju

Otkaz servera nakon prijema zahteva od klijenta



a) normalan rad sistema

Server može da otkáže u bilo kom trenutku

b) otkaz servera nakon izvršenja zahteva, a pre slanja odgovora

c) otkaz servera pre izvršenja zahteva

Problem je što različite akcije treba da se preduzmu u slučaju b) i c) a klijent ne može razlikovati slučaj b) i c) . U oba slučaja na strani klijenta će isteći time-out! U slučaju b) klijent treba da emituje grešku (expection), u slučaju c) samo da obavi retransmisiju

Dva rešenja su moguća:

1. klijent šalje zahteve sve dok ne dobije odgovor (at least one semantika – garantuje da će se RPC izvršiti bar jednom, a možda i više puta - primenljivo kod idempotentnih funkcija)
2. Odmah signalizirati grešku nakon isteka time-outa (at most one semantika – RPC će se izvršiti najviše jednom, a možda i ni jednom)

Primer: otkaz servera

- * najbolje bi bilo obezbediti tačno jedno izvršenje (exactly one semanika), ali nema načina da se u opštem slučaju to garantuje.
- * PRIMER:
 - pretpostavimo da udaljena operacija zahteva štampanje teksta na udaljenom printeru preko servera za štampanje
 - kada klijent pošalje zahtev, on prima potvrdu da je zahtev prosledjen serveru
 - server može poslati klijentu potvrdu pre nego što prosledi nalog štampaču ili nakon što je tekst odštampan.
 - Pretpostavimo da je server otkazao i zatim se oporavio od otkaza i obaveštava klijent da ponovo radi
 - problem je što klijent ne zna da li je njegov zahtev za štampanjem obavljen ili ne.
 - postoje 4 strategije koje klijent može da primeni:
 1. klijent može da odluči da nikada ne ponavlja zahtev, po cenu da tekst ne bude odštampan
 2. klijent uvek ponavlja zahtev, što može dovesti da tekst bude odštampan dva puta
 3. klijent će ponoviti zahtev ako nije primio potvrdu od servera
 4. klijent ponavlja zahtev samo ako je je primio potvrdu za prethodno izdati zahtev

Primer: otkaz servera (nast.)

* Na strani servera tri događaja mogu da nastupe:

- slanje poruke o okončanju zadatka (M)
- štampanje teksta (P)
- otkaz servera (C)

* Ovi događaji mogu da nastupe u 6 različitih redosleda:

1. $M \rightarrow P \rightarrow C$: otkaz servera se desio nakon slanja potvrde i štampanja teksta
2. $M \rightarrow C (\rightarrow P)$: otkaz servera se desio nakon slanja potvrde, a pre štampanja teksta
3. $P \rightarrow M \rightarrow C$: otkaz servera nastupio nakon štampanja i slanja potvrde
4. $P \rightarrow C (\rightarrow M)$: tekst odštampan, server otkazao pre slanja potvrde
5. $C (\rightarrow P \rightarrow M)$: server otkazao pre nego što je bilo šta uradio
6. $C (\rightarrow M \rightarrow P)$: server otkazao pre nego što je bilo šta uradio

* zagrade () ukazuju na događaje koji ne mogu da nastupe jer je server već otkazao

Primer: otkaz servera (nast.)

Različite kombinacije klijent i server strategija pri otkazu servera i rezultati štampanja: OK – štampano tačno jednom, DUP – duplirano, ZERO – ni jednom

Klijent		Server					
		Strategija M -> P		Strategija P -> M			
Ponavljjanje zahteva		MPC	MC(P)	C(MP)	PMC	PC(M)	C(PM)
uvek		DUP	OK	OK	DUP	DUP	OK
nikad		OK	ZERO	ZERO	OK	OK	ZERO
Samo ako je primljen ACK		DUP	OK	ZERO	DUP	OK	ZERO
Samo ako nije primljen ACK		OK	ZERO	OK	OK	DUP	OK

Ne postoji ni jedna kombinacija klijent i server strategije koja će garantovati da će se štampanje obaviti samo jednom za svaku kombinaciju događaja na server strani: klijent nikada ne može znati da li je tekst otštampan pre nego što je server otkazao ili ne

odgovor servera klijentu izgubljen

* Klijent može ponovo poslati zahtev serveru

- da li je bezbedno ponovo poslati zahtev?
 - kod idempotentnih zahteva nema problema
 - Npr: čitanje fajla
 - *kontraprimer*: transfer novca sa bankovnog računa

* Rešenje: dodeliti redne brojeve zahtevima

- Server vodi računa o rednim brojevima klijentskih zahteva i može otkriti duplikat i odbiti da izvrši zahtev ponovo
- ili, postaviti u zaglavlju poruke RETRANSMISSION bit kako bi se razlikovao originalni zahtev od retransmisije
 - ovo rešenje ne zahteva da server vodi računa o rednim brojevima zahteva klijenata
- u obe varijante server mora poslati ponovo poruku klijentu